

Index – Unit 2 Questions

Q#	Question Topic	Marks	Status
Q1	Framing Techniques – 4 Types + Bit Stuffing Numerical	10M	■ Repeat
Q2	CRC – Complete Numerical (1101011011 ÷ 10011)	10M	■ Every Year
Q3	Hamming Code – Error Detection + Correction Numerical	10M	■ 2022 PYQ
Q4	Stop-and-Wait ARQ + Efficiency Numerical	5-10M	■ Repeat
Q5	Go-Back-N ARQ + Window Size + Sequence Numbers	10M	■ Repeat
Q6	Selective Repeat ARQ + Comparison Table	10M	■ Repeat
Q7	Sliding Window Protocol Numerical	10M	■ PYQ Confirmed
Q8	Pure ALOHA + Slotted ALOHA + Numerical	10M	■ 2022 PYQ
Q9	CSMA/CD + CSMA/CA – Working + Comparison	10M	High
Q10	Network Devices – Hub, Switch, Bridge, Router, Repeater	5M	■ 2023 PYQ
Q11	Piggybacking kya hai?	2M	■ 2022 PYQ

Q1. Framing Techniques + Bit Stuffing

Marks: 10M | ■ Repeat

Q1.

[10M | ■ Repeat]

Framing kya hai? Data Link Layer mein framing ke 4 techniques explain karo. Bit stuffing ka numerical example bhi solve karo.

Framing kya hai?

Data Link Layer Network Layer se data (packets) receive karta hai aur unhe **frames** mein encapsulate karta hai. Framing ek mechanism hai jisse receiver identify kar sake ki ek frame kahan shuru hoti hai aur kahan khatam. Bina framing ke receiver ko nahi pata kab ek frame khatam hui aur doosri shuru hui.

4 Framing Techniques:

1. Character Count (Byte Count)

Frame ki pehli field mein total characters/bytes ki count hoti hai. Receiver count padh ke jaanta hai frame kab khatam hogi.

Problem: Agar count field mein transmission error ho jaaye, receiver sync kho deta hai aur baaki saari frames galat interpret hoti hain. Isliye practically use nahi hota.

2. Flag Bytes with Byte Stuffing (Character-Oriented)

Ek special flag byte (e.g. 01111110) frame ke start aur end pe lagayi jaati hai. Agar data mein yahi pattern aaye, ek escape byte (ESC) pehle stuff ki jaati hai.

Example: Data: A ESC B → Transmit: FLAG ESC A ESC ESC B FLAG

Byte Stuffing Rule: Data mein FLAG → add ESC before it; Data mein ESC → add ESC before it

Used in: PPP (Point-to-Point Protocol)

3. Flag Bits + Bit Stuffing (Bit-Oriented)

Flag sequence **01111110** frame boundaries mark karti hai. Data mein agar 5 consecutive 1s aayein, sender automatically ek 0 insert kar deta hai (bit stuffing). Receiver 5 ek ke baad ek 0 remove kar deta hai.

Example (original data): 0110111111111110010

After stuffing (add 0 after 5 ones): 0110111110111110110010

Used in: HDLC, PPP bit mode, SONET

4. Physical Layer Coding Violations

Physical layer mein kuch special signal patterns use kiye jaate hain jo normal data mein appear nahi karte. Ye patterns frame boundaries mark karte hain.

Example:	Manchester encoding mein H→H ya L→L (no transition) invalid signal hai, use frame delimiter ke roop mein use karte hain.
-----------------	--

Used in:	Ethernet, Token Ring
-----------------	----------------------

Bit Stuffing Solved Example:

Original Data: 01101111111110100 **Rule:** After every 5 consecutive 1s → insert a 0 **Step by step:**
Original: 0 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 0 0 Count 1s: 1st run of 5 ones found at position 6-10 → insert 0
After 1st: 0 1 1 0 1 1 1 1 1 1 0 1 1 1 0 1 0 0 Count again: next run 2nd group of 5 ones at new positions
→ insert 0 Final: 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 0 **At Receiver:** After receiving, remove every 0 that comes after 5 consecutive 1s → original data restored.

Q2. CRC – Cyclic Redundancy Check

Marks: 10M | ■ Every Year –
GUARANTEED

Q2.

[10M | Every Year]

CRC kya hai? CRC calculate karo: Data = 1101011011, Generator = 10011. Transmitted frame kya hogi? Receiver error kaise detect karta hai?

CRC kya hai?

CRC (Cyclic Redundancy Check) ek **error detection technique** hai jo binary division use karta hai. Sender data pe division karta hai aur remainder (FCS – Frame Check Sequence) data ke saath attach karta hai. Receiver same division karta hai — agar remainder 0 aaye toh koi error nahi, nahi toh error detect hui.

CRC Steps (Sender Side):

1. Agar generator ka degree n hai (generator mein $n+1$ bits), toh data ke end mein n zeros append karo.
2. Augmented data ko generator se binary division (XOR-based) karo.
3. Jo remainder aaye — wo hai CRC/FCS.
4. Original data ke end mein ye CRC append karo. Ye transmitted frame hai.

Given: Data (M) = 1101011011 **Generator (G)** = 10011 (degree = 4, so append 4 zeros to data)

Step 1: Append 4 zeros → Augmented Data:

$M' = 1101011011 \ 0000$ (14 bits total)

Step 2: Binary Division (XOR) – $M' \div G$:

```
1101011011 0000 ← dividend 10011 ← divisor ████████████████████████████████  
1101011011 0000 10011 ██████ 100111 (XOR result: 1101 XOR 1001 = 0100, bring down  
next) Full long division (XOR at each step): 1101011011 0000 ÷ 10011  
████████████████████████████████████████ 10011 → 1101 XOR 1001 = 0100 → bring 1 → 1001 1001 1 →  
XOR 10011 → 00000 → bring 0 → 00000 00001 → bring 1 → 00011 00011 → bring 0 →  
00110 00110 → bring 1 → 01101 01101 → bring 1 → 11011 11011 → XOR 10011 → 01000 →  
bring 0 → 10000 10000 → XOR 10011 → 00011 → bring 0 → 00110 00110 → bring 0 →  
01100 01100 → XOR 10011 → 11111 (< 5 bits, this is remainder) REMAINDER (CRC) = 1110
```

Step 3: Transmitted Frame:

CRC = 1110 Transmitted Frame = Data + CRC = 1101011011 + 1110 = 11010110111110

Receiver Side – Error Detection:

- Received frame ko generator (10011) se divide karo.
- Agar remainder = 0000 → **No error detected**, data accept karo.
- Agar remainder $\neq$ 0000 → **Error detected**, frame discard karo (retransmit request).

Important Properties of CRC:

- Detects all single-bit errors
- Detects all double-bit errors (with proper generator)
- Detects all odd number of errors
- Detects all burst errors of length $\leq n$ (degree of generator)
- CRC does NOT correct errors — only detects
- Common generators: CRC-8, CRC-16, CRC-32 (Ethernet uses CRC-32)

Q3. Hamming Code – Error Detection + Correction

Marks: 10M | ■ PYQ 2022

Q3.

[10M | ■
PYQ
2022]

Hamming Code kya hai? 7-bit received codeword 1010101 mein error detect karo aur correct karo. Even parity ke saath 4-bit data 1011 ka Hamming code construct karo.

Hamming Code kya hai?

Hamming Code R.W. Hamming ne 1950 mein develop kiya. Ye ek **Single Error Correcting, Double Error Detecting (SECDED)** code hai. Data bits ke beech mein special **parity bits** (redundant bits) insert kiye jaate hain jo error ki exact position indicate karte hain.

Redundant Bits count kaise karte hain:

Agar data bits = d hain, redundant bits r chahiye jahan: $2^r \geq d + r + 1$

$d=4$ ke liye: $r=3$ try karo $\rightarrow 2^3=8 \geq 4+3+1=8 \checkmark \rightarrow r=3$ redundant bits chahiye Total bits in codeword = $d + r = 4 + 3 = 7$ bits

Part A – Hamming Code Construct karo for data 1011 (even parity)

Data bits ($d_1 d_2 d_3 d_4$) = 1 0 1 1 Parity bits at positions 1, 2, 4 (powers of 2) Positions: 1 2 3 4 5 6 7
Type: P1 P2 D1 P4 D2 D3 D4 Data fill: ? ? 1 ? 0 1 1

Parity bit calculation (even parity = XOR of all covered positions):

P1 (pos 1): covers positions 1,3,5,7 $\rightarrow$ bits at 3,5,7 = $D_1, D_2, D_4 = 1, 0, 1$ $P_1 = 1 \text{ XOR } 0 \text{ XOR } 1 = 0$
P2 (pos 2): covers positions 2,3,6,7 $\rightarrow$ bits at 3,6,7 = $D_1, D_3, D_4 = 1, 1, 1$ $P_2 = 1 \text{ XOR } 1 \text{ XOR } 1 = 1$
P4 (pos 4): covers positions 4,5,6,7 $\rightarrow$ bits at 5,6,7 = $D_2, D_3, D_4 = 0, 1, 1$ $P_4 = 0 \text{ XOR } 1 \text{ XOR } 1 = 0$

Positions: 1 2 3 4 5 6 7 Codeword: 0 1 1 0 0 1 1 Transmitted Hamming Codeword = 0110011

Part B – Error Detection + Correction in received codeword 1010101

Received codeword = 1 0 1 0 1 0 1

Positions: 1 2 3 4 5 6 7 Received: 1 0 1 0 1 0 1

Check parity bits (recalculate using received bits):

C1 (check pos 1): covers 1,3,5,7 $\rightarrow 1, 1, 1, 1 \rightarrow \text{XOR} = 0$ (EVEN $\rightarrow C_1=0$)
C2 (check pos 2): covers 2,3,6,7 $\rightarrow 0, 1, 0, 1 \rightarrow \text{XOR} = 0$ (EVEN $\rightarrow C_2=0$)
C4 (check pos 4): covers 4,5,6,7 $\rightarrow 0, 1, 0, 1 \rightarrow \text{XOR} = 0$ (EVEN $\rightarrow C_4=0$)

Error Syndrome = $C_4 C_2 C_1 = 000$

Syndrome = 000 → **No error detected!** Codeword 1010101 is valid — no single-bit error present. **If syndrome was non-zero:** Example: Syndrome = 011 (binary) = 3 (decimal) → Error at position 3
Correction: Flip the bit at position 3 → error corrected!

Error Position Table (for 7-bit Hamming code):

Syndrome (C4 C2 C1)	Error Position	Action
000	No error	Accept codeword
001	Position 1 (P1)	Flip bit at position 1
010	Position 2 (P2)	Flip bit at position 2
011	Position 3 (D1)	Flip bit at position 3
100	Position 4 (P4)	Flip bit at position 4
101	Position 5 (D2)	Flip bit at position 5
110	Position 6 (D3)	Flip bit at position 6
111	Position 7 (D4)	Flip bit at position 7

Q4. Stop-and-Wait ARQ + Efficiency

Marks: 5-10M | ■ Repeat

Q4.

[10M | ■ Repeat]

Stop-and-Wait ARQ kya hai? Working explain karo. Efficiency formula derive karo aur numerical solve karo: Bandwidth=1Mbps, Frame=1000bits, Propagation delay=270ms.

Stop-and-Wait ARQ kya hai?

Stop-and-Wait ARQ (Automatic Repeat reQuest) mein sender ek frame transmit karta hai aur phir **rukta hai (stops)** — ACK (acknowledgment) ka wait karta hai. ACK milne ke baad hi next frame transmit karta hai. Agar ACK nahi mila (timeout), frame retransmit hoti hai.

Working – Normal Case (No Error):

- Sender Frame 0 transmit karta hai.
- Frame receiver tak pahunchi → receiver ACK 0 bhejta hai.
- Sender ACK 0 receive karta hai → Frame 1 transmit karta hai.
- Aise karta rehta hai → alternating 0 aur 1 sequence numbers.

Working – Error Case:

- Frame lost/corrupted → receiver NAK bhejta hai ya kuch nahi bhejta.
- Sender ka timer expire hota hai (timeout) → same frame retransmit.
- ACK lost → sender timeout → retransmit → receiver duplicate detect karta hai (sequence number se) → discard karta hai, ACK dobara bhejta hai.

Efficiency Formula Derivation:

Let: T_t = Transmission time, T_p = Propagation delay, $a = T_p/T_t$

Total time for 1 frame cycle = $T_t + 2T_p + T_{ack} \approx T_t + 2T_p$ ($T_{ack} \approx 0$)

Efficiency η = Useful time / Total time = $T_t / (T_t + 2T_p) = 1 / (1 + 2a)$

where $a = T_p/T_t$ $\eta = 1/(1+2a)$ ← THIS FORMULA YAAD KARO

Numerical: Bandwidth = 1 Mbps, Frame size = 1000 bits, Prop. delay = 270 ms T_t = Frame size / Bandwidth = $1000 / 1,000,000 = 1 \text{ ms}$ $T_p = 270 \text{ ms}$ $a = T_p / T_t = 270 / 1 = 270$ $\eta = 1 / (1 + 2 \times 270) = 1 / 541 = 0.00185 = 0.185\%$ **Bahut poor efficiency!** 99.8% time mein sender idle hai — wait kar raha hai. Iska solution hai: Sliding Window / Pipelining protocols.

Stop-and-Wait Limitations:

- Extremely low efficiency for high-delay links (satellites, WAN)
- Only 1 frame in transit at a time → bandwidth waste
- Solution: Go-Back-N aur Selective Repeat (pipelining)

Q5 & Q6. Go-Back-N + Selective Repeat ARQ

Marks: 10M each | ■ Repeat

Q5.

[10M | ■ Repeat]

Go-Back-N ARQ aur Selective Repeat ARQ explain karo. Window size, sequence numbers, aur comparison table ke saath.

Pipelining kya hai?

Pipelining mein sender ACK ka wait kiye bina **multiple frames transmit** karta rehta hai. Isse channel busy rehta hai aur efficiency badh jaati hai. Window size (W) = ek time pe maximum kitni frames outstanding ho sakti hain.

Go-Back-N (GBN) ARQ:

GBN mein agar koi frame lost ya corrupted ho, toh us frame se lekar **saari baad wali frames dobara transmit** ki jaati hain — chahe wo sahi pahunchi ho.

Window Size: $W \leq 2^n - 1$ where n = sequence number bits

Receiver Buffer: Sirf 1 frame — order mein hi accept karta hai

Error action: Error frame + saari subsequent frames discard, NAK bhejo

Efficiency: $\eta = W/(1+2a)$ if $W < (1+2a)$, else $\eta = 1$

GBN Example: W=4, frames 0,1,2,3,4,5,6,7 send karne hain, Frame 2 lost Send: 0 1 2 3 → Frame 2 lost → receiver 3 ko discard karta hai, NAK 2 bhejta hai Retransmit: 2 3 4 5 (go back to frame 2, send all again) **Wasteful!** Frame 3 sahi tha lekin dobara bhejna pada.

Selective Repeat (SR) ARQ:

SR mein sirf wo frame retransmit hoti hai jo **actually lost/corrupted** hui thi. Receiver out-of-order frames ko buffer mein store karta hai.

Window Size: $W \leq 2^{n-1}$ (HALF of total sequence numbers)

Receiver Buffer: W frames — out-of-order frames store kar sakta hai

Error action: Sirf error wali frame retransmit — doosri OK frames accept

Efficiency: $\eta = W/(1+2a)$ if $W < (1+2a)$, else $\eta = 1$

SR Example: W=4, Frame 2 lost Send: 0 1 2 3 → Frame 2 lost → receiver 3 buffer mein store karta hai Retransmit: only Frame 2 Receiver 2 milne ke baad: 2,3 ko sequence mein de deta hai network layer ko **Efficient!** Sirf lost frame retransmit.

Comparison Table – Stop-and-Wait vs GBN vs SR:

Feature	Stop-and-Wait	Go-Back-N	Selective Repeat
Window size (sender)	1	$2^n - 1$	2^{n-1}

Window size (receiver)	1	1	2^{n-1}
Seq numbers needed	2	2^n	2^n
Retransmission	Only lost frame	Lost + all after	Only lost frame
Receiver buffer	1 frame	1 frame	W frames
Complexity	Simplest	Medium	Most complex
Efficiency	Lowest ($1/(1+2a)$)	Medium	Highest
Bandwidth use	Worst	Better	Best
n=3 bits, W=?	W=1	$W \leq 7$	$W \leq 4$

Q7. Sliding Window Protocol – Numericals

Marks: 10M | ■ PYQ Confirmed

Q7.

[10M | ■
PYQ Confirmed]

Sliding Window numerical: (i) $T_t=1\text{ms}$, $T_p=24\text{ms}$ — minimum window size for 100% efficiency? (ii) GBN-3: 10 packets, har 5th packet lost — total transmissions?

Key Formulas:

$a = T_p / T_t$ For 100% efficiency: Window size $W \geq 1 + 2a$ (pipeline must be full) Efficiency with window W : $\eta = W / (1+2a)$ if $W < (1+2a)$ $\eta = 1$ if $W \geq (1+2a)$

Numerical 1: $T_t = 1\text{ms}$, $T_p = 24\text{ms}$, minimum window size?

$a = T_p / T_t = 24 / 1 = 24$ For 100% efficiency: $W \geq 1 + 2a = 1 + 2 \times 24 = 1 + 48 = 49$ Minimum window size $W = 49$ frames Sequence numbers required: At least 49 $\rightarrow$ use n bits where $2^n \geq 49 \rightarrow n = 6$ bits ($2^6 = 64 \geq 49$)

Numerical 2: GBN with window size 3, 10 packets, every 5th packet lost

Packets to send: 1,2,3,4,5,6,7,8,9,10 (every 5th lost means: P5, P10 lost) Transmission trace with $W=3$:
Send: 1,2,3 $\rightarrow$ ACK 1,2,3 received $\rightarrow$ Send 4,5,6 Packet 5 lost $\rightarrow$ receiver discards 6, sends NAK 5
Retransmit: 5,6,7 $\rightarrow$ ACK 5,6,7 $\rightarrow$ Send 8,9,10 Packet 10 lost $\rightarrow$ receiver discards nothing (last), NAK 10
Retransmit: 10 (only one left) Transmissions count: Original: 10 packets Retransmit P5 + P6 (GBN goes back): 2 extra Retransmit P10: 1 extra **Total transmissions = 10 + 2 + 1 = 13**

Numerical 3: Efficiency = at least 50% (Stop-and-Wait)

$\eta = 1/(1+2a) \geq 0.5 \rightarrow 1 + 2a \leq 2 \rightarrow 2a \leq 1 \rightarrow a \leq 0.5 \rightarrow T_p/T_t \leq 0.5 \rightarrow T_p \leq T_t/2$ Matlab: Propagation delay, Transmission delay ke half se zyada nahi honi chahiye for Stop-and-Wait to achieve at least 50% efficiency.

Q8. Pure ALOHA + Slotted ALOHA

Marks: 10M | ■ PYQ 2022

Q8.

[10M | ■
PYQ
2022]

Pure ALOHA aur Slotted ALOHA explain karo. Throughput formula, maximum efficiency. Numerical: Slotted ALOHA mein 10% slots idle hain, channel load kya hai?

ALOHA kya hai?

ALOHA ek random access protocol hai jo University of Hawaii ne 1970 mein develop kiya. Multiple stations ek shared channel use karti hain bina kisi central control ke. Collision ho sakti hai — collision ke baad random time wait karke retransmit kiya jaata hai.

Pure ALOHA:

- Koi bhi station **jab chahein** transmit kar sakti hai.
- Collision hone par: random time wait karo, phir retransmit.
- **Vulnerable period** = $2 \times \text{frame transmission time}$ ($2T_t$)
- **Throughput** $S = G \times e^{-2G}$
- Maximum throughput at $G=0.5$: **$S_{\text{max}} = 1/(2e) \approx 18.4\%$**

Slotted ALOHA:

- Time discrete slots mein divided hai (slot size = frame transmission time).
- Station sirf **slot boundary pe transmit** kar sakti hai.
- Isse vulnerable period HALF ho jaata hai = $1 \times \text{frame transmission time}$ (T_t)
- **Throughput** $S = G \times e^{-G}$
- Maximum throughput at $G=1$: **$S_{\text{max}} = 1/e \approx 36.8\%$**

Pure ALOHA vs Slotted ALOHA Comparison:

Feature	Pure ALOHA	Slotted ALOHA
Transmission time	Any time	Only at slot boundaries
Synchronization	Not required	Required (clocks sync)
Vulnerable period	$2 \times T_t$	$1 \times T_t$
Throughput formula	$S = G \times e^{-2G}$	$S = G \times e^{-G}$
Max throughput	18.4% (at $G=0.5$)	36.8% (at $G=1.0$)
Complexity	Simpler	More complex
Collision probability	Higher	Lower (halved)

Numerical: Slotted ALOHA mein 10% slots idle hain, G kya hai? Idle slot probability = $P(0 \text{ transmissions in slot}) = e^{-G}$ 10% slots idle $\rightarrow P(\text{idle}) = 0.10 \rightarrow e^{-G} = 0.10 \rightarrow -G = \ln(0.10) = -2.303 \rightarrow \mathbf{G = 2.303}$ Throughput $S = G \times e^{-G} = 2.303 \times 0.10 = \mathbf{0.2303 = 23.03\%}$

Q9. CSMA/CD + CSMA/CA

Marks: 10M | High

Q9.

[10M | ■
High]

CSMA/CD aur CSMA/CA explain karo — working, difference, aur kahan use hota hai. 1-persistent, non-persistent, p-persistent CSMA bhi explain karo.

CSMA – Carrier Sense Multiple Access:

CSMA mein station transmit karne se pehle channel **sense** karta hai (check karta hai koi transmit to nahi kar raha). "Listen before talk" principle.

1-Persistent CSMA Channel busy → wait karo aur continuously sense karo. Jab channel free ho → immediately transmit karo (probability 1). *Issue: Aggressive — multiple stations simultaneously transmit karte hain jab channel free hota hai → high collision*

Non-Persistent CSMA Channel busy → random time wait karo, phir dobara sense karo. Channel free → transmit. *Issue: Less aggressive — less collision but channel may be idle unnecessarily*

p-Persistent CSMA Channel free → probability p se transmit karo, probability (1-p) se next slot tak wait karo. Used in slotted channels. *Issue: Balance between 1-persistent and non-persistent*

CSMA/CD – Collision Detection (Ethernet):

CD = Collision Detection. Station transmit karte waqt channel monitor karta rehta hai. Collision detect hone par immediately stop karta hai aur jam signal bhejta hai.

Step 1: Channel sense karo. Free hai → transmit shuru karo.

Step 2: Transmit karte waqt continuously monitor karo.

Step 3: Collision detect? → Jam signal bhejo (32 bits) → stop transmission.

Step 4: Random backoff time wait karo (Binary Exponential Backoff algorithm).

Step 5: Dobara transmit karo.

■ ■ Important Formula

Minimum Frame Size in Ethernet: Frame transmission time $\geq 2 \times$ Propagation delay → $T_t \geq 2T_p$ (taaki sender collision detect kar sake before finishing) → Minimum frame size = 64 bytes (Ethernet standard)

CSMA/CA – Collision Avoidance (Wi-Fi/802.11):

CA = Collision Avoidance. Wireless mein collision detect karna possible nahi (hidden terminal problem) isliye **avoid** karte hain.

IFS (Inter-Frame Space): Channel free hone ke baad bhi kuch time wait karo.

RTS/CTS: Request-to-Send / Clear-to-Send handshake — channel reserve karo before data.

Random Backoff: Multiple stations simultaneously transmit na karein — random wait.

ACK: Har unicast frame ka ACK chahiye (wireless unreliable).

CSMA/CD vs CSMA/CA Comparison:

Feature	CSMA/CD	CSMA/CA
Full form	Collision Detection	Collision Avoidance
Used in	Wired Ethernet (802.3)	Wireless Wi-Fi (802.11)
Collision handling	Detect and retransmit	Avoid before transmitting
Channel monitoring	During transmission	Before transmission
ACK required?	No (wired reliable)	Yes (wireless unreliable)
Hidden terminal	Not an issue	Solved using RTS/CTS
Efficiency	Higher	Lower (overhead)
Min frame size	64 bytes (Ethernet)	No fixed minimum

Q10. Network Devices

Marks: 5M | ■ 2023 PYQ

Q10.

[5M | ■
2023
PYQ]

Network devices explain karo: Hub, Switch, Bridge, Router, Repeater — OSI layer, kaam, aur differences.

Device	OSI Layer	Working	Key Feature	Example
Repeater	Layer 1 (Physical)	Signal amplify/regenerate karta hai	Signal regeneration only	Canal Extender
Hub	Layer 1 (Physical)	Data sab ports pe broadcast karta hai	No filtering, creates collision domain	Old Ethernet hub
Bridge	Layer 2 (Data Link)	MAC addresses ke basis pe frames filter karta hai	Resolves collision domain	Wireless bridge, VLAN controller
Switch	Layer 2 (Data Link)	MAC address table maintain karta hai	Each port has separate collision domain	Cisco Catalyst, HP ProCurve, Mikrotik
Router	Layer 3 (Network)	IP addresses se packets forward karta hai	Separates broadcast domains	ISA Firewall, Cisco IOS
Gateway	Layer 4-7 (All)	Protocol translation — different networks ko connect karta hai	Controls between different protocols	Intelligent proxy, NAT gateway

Hub vs Switch (Most Common 2M Q):

- Hub: Layer 1, broadcasts to all ports, single collision domain, half-duplex
- Switch: Layer 2, forwards only to destination, separate collision domain per port, full-duplex

Router vs Switch:

- Switch: Layer 2, same network, MAC-based, fast
- Router: Layer 3, different networks, IP-based, intelligent routing

Q11. Piggybacking

Marks: 2M | ■ PYQ 2022

Q11.

[2M | ■
PYQ
2022]

Piggybacking kya hota hai? Data Link Layer mein iska use kaise hota hai?

Piggybacking kya hai?

Piggybacking ek technique hai jisme **ACK (acknowledgment)** ko **incoming data ke saath attach** karke bheja jaata hai — separately nahi. Jab ek station doosri station se data receive karti hai aur usse apna data bhi bhejna hota hai, toh ACK apne data frame ke header mein piggyback (saath mein) karke bhejti hai. Isse bandwidth bachti hai.

Example (without piggybacking): A → B: Data Frame 0 B → A: ACK 0 (separate frame) B → A: Data Frame 0 A → B: ACK 0 (separate frame) **Example (with piggybacking):** A → B: Data Frame 0 B → A: Data Frame 0 + ACK 0 (combined in one frame!) A → B: Data Frame 1 + ACK 0 (combined again)

Advantage: ACK ke liye alag frame bhejne ki zaroorat nahi → bandwidth save **Used in:** TCP (ACK is piggybacked in data segments), HDLC

Quick Revision – Unit 2 Formula Sheet

Exam se pehle zaroor padhna

Topic	Key Formula / Key Point
CRC	Append n zeros (n=degree of G) → Binary divide → Remainder = CRC → Append to data
CRC Check	Divide received frame by G → Remainder = 0 → No error; ≠ 0 → Error
Hamming bits r	$2^r \geq d + r + 1$; $d=4 \rightarrow r=3$; total bits = $d+r = 7$
Parity positions	Powers of 2: 1, 2, 4, 8, 16... (positions in codeword)
Hamming syndrome	C4C2C1 binary = error position; 000 = no error
Stop-and-Wait η	$\eta = 1/(1+2a)$; $a = T_p/T_t$
Window for 100%	$W \geq 1+2a \rightarrow W_{min} = \text{ceil}(1+2a)$
Sliding Window η	$\eta = W/(1+2a)$ if $W < 1+2a$; else $\eta = 1$
GBN window size	$W \leq 2^r - 1$ (sender and receiver window)
SR window size	$W \leq 2^{r-1}$ (both sender and receiver)
Pure ALOHA	$S = G \times e^{-2G}$; Max S = 18.4% at $G=0.5$
Slotted ALOHA	$S = 2G \times e^{-G}$; Max S = 36.8% at $G=1.0$
Slotted vs Pure	Slotted = 2× efficiency; vulnerable period = T_t not $2T_t$
CSMA/CD min frame	$T_t \geq 2T_p \rightarrow \text{Frame size} \geq 2 \times \text{Propagation delay} \times \text{Bandwidth}$
Hub vs Switch	Hub=Layer1,broadcast; Switch=Layer2,unicast,MAC table
Bridge vs Router	Bridge=Layer2,MAC,same subnet; Router=Layer3,IP,diff subnets
Piggybacking	ACK attached to outgoing data frame – saves bandwidth
Bit stuffing	After 5 consecutive 1s → insert 0; receiver removes it
Byte stuffing	FLAG in data → add ESC; ESC in data → add ESC before it

■ FINAL EXAM TIPS – Unit 2

Exam Strategy – Unit 2 ke liye:

- CRC numerical GUARANTEED: $1101011011 \div 10011 \rightarrow$ steps clearly dikhao, remainder = CRC
- Hamming Code: r bits calculate karo → positions assign karo → parity calculate karo → error syndrome find karo
- Stop-and-Wait efficiency: $\eta = 1/(1+2a)$ formula + numerical ($T_t=1\text{ms}$, $T_p=24\text{ms} \rightarrow 0.185\%$)
- GBN vs SR: Window size formulas aur comparison table format mein likho
- ALOHA: Pure (18.4%) vs Slotted (36.8%) max throughput + numerical
- Network devices: Table format mein OSI layer + key feature + example likho